

BUDDMEYER - PDA

Integração de Autenticação

Documentação Técnica de API – PDAAuthLogin

Versão: 1.0

Data: 28/01/2026

Autor: Caique Silva

Classificação: Equipe Técnica Personalitec

Sumário

Sumário	2
1. Visão Geral	3
1.1 Responsabilidades da Classe	3
1.2 Dependências e Configurações	3
2. Endpoints da API	4
2.1 POST /api/Autenticacao	4
Parâmetros de Entrada (Body – JSON)	4
Exemplo de Requisição	4
Parâmetros de Retorno (Body – JSON)	4
Exemplo de Resposta	4
2.2 POST /api/Autenticacao/Refresh-Token	5
Parâmetros de Entrada (Body – JSON)	5
Exemplo de Requisição	5
Parâmetros de Retorno (Body – JSON)	5
Exemplo de Resposta	6
3. Métodos da Classe PDAAuthLogin	7
3.1 Construtor – New()	7
3.2 GetToken()	7
3.3 GetHeader()	7
3.4 RefreshToken()	8
3.5 IsAuthenticated()	8
3.6 GetLastError()	8
4. Exemplos de Uso	9
4.1 Autenticação básica e consumo de outro endpoint	9
4.2 Renovação de Token	9
4.3 Uso em Rotina com Tratamento de Erros	9
5. Erros e Tratativas	11
5.1 Tabela de Erros Conhecidos	11
5.2 Fluxo de Tratamento Recomendado	11
5.3 Boas Práticas	11
6. Atributos Internos da Classe	12
7. Histórico de Versões	13

1. Visão Geral

A classe PDAAuthLogin é responsável por gerenciar toda a autenticação com a API do PDA HUB dentro do ambiente Protheus (TOTVS). Ela encapsula o ciclo de vida do token de acesso, desde a obtenção inicial até a renovação automática, expondo uma interface simples para que outras classes e rotinas do sistema possam se autenticar com segurança.

Esta documentação destina-se à equipe técnica (desenvolvedores e analistas) que precisam integrar ou manter rotinas que consomem os serviços do PDA HUB.

1.1 Responsabilidades da Classe

- Autenticar o usuário via credenciais (login/senha) configuradas nos parâmetros do sistema.
- Armazenar e expor o Bearer Token (accessToken) para uso nos cabeçalhos HTTP das requisições.
- Renovar o token de forma controlada via refreshToken, sem necessidade de nova autenticação completa.
- Sinalizar o estado de autenticação e registrar erros para diagnóstico.

1.2 Dependências e Configurações

A classe depende dos seguintes parâmetros do Protheus (MV_PAR):

Parâmetro	Descrição	Exemplo
BUD_1505LG	Login de acesso à API PDA HUB	usuario.api
BUD_1505PW	Senha do usuário da API	SenhaSegura123!
BUD_1505UR	URL base da API PDA HUB	https://stg.api.pdahub.com.br

Esses parâmetros devem ser cadastrados no Configurador (SIGACFG) antes da utilização da classe.

2. Endpoints da API

2.1 POST /api/Autenticacao

Método	POST
Endpoint	/api/Autenticacao
Descrição	Realiza a autenticação com login e senha. Retorna um par de tokens (accessToken + refreshToken).

Parâmetros de Entrada (Body – JSON)

Campo	Tipo	Obrigatório	Descrição
login	string	Sim	Login de acesso do usuário na API
password	string	Sim	Senha do usuário

Exemplo de Requisição

```
POST /api/Autenticacao
Content-Type: application/json
```

```
{
  "login": "usuario.api",
  "password": "SenhaSegura123!"
}
```

Parâmetros de Retorno (Body – JSON)

Campo	Tipo	Descrição
authenticated	boolean	Indica se a autenticação foi realizada com sucesso
accessToken	string	Bearer Token (formato JWT) a ser enviado no header Authorization das requisições
refreshToken	string (UUID)	Token para renovação sem nova autenticação completa
created	string (datetime)	Data/hora de criação do token (ISO 8601)
expiration	string (datetime)	Data/hora de expiração do token (ISO 8601)

Exemplo de Resposta

```
{
  "authenticated": true,
  "created": "2023-10-06T13:01:06.6233477+00:00",
  "expiration": "2023-10-06T15:01:06.62335+00:00",
}
```

```
{  "accessToken":  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  "refreshToken":  "0104f8ec-5752-40d1-997c-9fe7af8d9872"}
```

2.2 POST /api/Autenticacao/Refresh-Token

Método	POST
---------------	------

Endpoint	/api/Autenticacao/Refresh-Token
-----------------	---------------------------------

Descrição	Renova o accessToken utilizando o refreshToken atual. Evita nova autenticação completa enquanto o refreshToken for válido.
------------------	--

Parâmetros de Entrada (Body – JSON)

Campo	Tipo	Obrigatório	Descrição
login	string	Sim	Login do usuário (mesmo da autenticação inicial)
refreshToken	string	Sim	Token de renovação obtido na autenticação

Exemplo de Requisição

```
POST /api/Autenticacao/Refresh-Token
Content-Type: application/json

{
  "login":      "usuario.api",
  "refreshToken": "0104f8ec-5752-40d1-997c-9fe7af8d9872"
}
```

Parâmetros de Retorno (Body – JSON)

O retorno do Refresh-Token é mais completo que o login inicial e inclui dados do usuário:

Campo	Tipo	Descrição
authenticated	boolean	Indica se a renovação foi bem-sucedida
accessToken	string	Novo Bearer Token (formato JWT) para uso nas requisições
refreshToken	string (UUID)	Novo refresh token (substitui o anterior)
created	string (datetime)	Data/hora de criação do novo token (ISO 8601)
expiration	string (datetime)	Data/hora de expiração do novo token (ISO 8601)
codigoUsuario	integer	Código interno do usuário na API PDA

Campo	Tipo	Descrição
login	string	Login do usuário autenticado
nome	string	Nome completo do usuário
email	string	E-mail do usuário
ativo	boolean	Indica se o usuário está ativo
alterarSenha	boolean	Indica se o usuário precisa alterar a senha
codigoPerfil	integer	Código do perfil de acesso do usuário
descricaoPerfil	string	Descrição do perfil de acesso
lider	boolean	Indica se o usuário é líder
clientes	array	Lista de clientes vinculados (campo: descricaoCliente)
usuInc / datInc	int / datetime	Usuário e data de inclusão do registro
usuAlt / datAlt	int / datetime	Usuário e data da última alteração

Exemplo de Resposta

```
{
  "authenticated": true,
  "created": "2023-10-06T14:22:01.6294887+00:00",
  "expiration": "2023-10-06T16:22:01.6294912+00:00",
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "0104f8ec-5752-40d1-997c-9fe7af8d9872",
  "codigoUsuario": 108,
  "login": "0000020",
  "nome": "Adm Inti Brand",
  "email": "usuario@empresa.com.br",
  "ativo": true,
  "alterarSenha": false,
  "codigoPerfil": 56,
  "descricaoPerfil": "Adm",
  "lider": true,
  "clientes": [{ "descricaoCliente": "Inti Brand" }]
}
```

3. Métodos da Classe PDAuthLogin

A tabela abaixo resume todos os métodos públicos disponíveis:

Método	Retorno	Descrição
New(cLogin, cPassword)	Self (objeto)	Construtor. Inicializa a classe e chama GetToken() automaticamente
GetToken()	string	Faz POST em /api/Autenticacao e armazena o Bearer Token obtido
GetHeader()	array	Retorna array com Content-Type e Authorization: Bearer <accessToken>
RefreshToken()	.T. / .F.	Renova o token via Refresh-Token. Retorna .T. em sucesso
IsAuthenticated()	.T. / .F.	Indica se a autenticação está ativa e válida
GetLastError()	string	Retorna a mensagem do último erro registrado

3.1 Construtor – New()

Assinatura: New(cLogin, cPassword)

Os parâmetros cLogin e cPassword são opcionais. Quando omitidos, a classe lê automaticamente os valores dos MV_PAR BUD_1505LG e BUD_1505PW. O construtor chama GetToken() internamente, de modo que o objeto já nasce autenticado se as credenciais forem válidas.

```
// Uso com parâmetros do sistema (padrão)
Local oAuth := PDAuthLogin():New()

// Uso com credenciais explícitas
Local oAuth := PDAuthLogin():New("usuario.api", "SenhaSegura123!")
```

3.2 GetToken()

Realiza a requisição POST ao endpoint de autenticação e persiste internamente: cToken (Bearer Token), cRefreshToken, cCreated, cExpiration e lAuthenticated. Retorna a string do accessToken ou vazio em caso de falha.

O accessToken segue o formato JWT (JSON Web Token) e deve ser enviado nos headers das demais requisições como: Authorization: Bearer <accessToken>.

3.3 GetHeader()

Monta e retorna um array de cabeçalhos prontos para uso em chamadas FwRest:

```
// Retorno esperado:  
{ "Content-Type: application/json",  
  "Authorization: Bearer eyJhbGci..." }
```

3.4 RefreshToken()

Deve ser chamado quando o accessToken expirar mas o refreshToken ainda for válido. Atualiza todos os tokens internos e o aHeader automaticamente. Retorna .T. se bem-sucedido.

3.5 IsAuthenticated()

Retorna o valor lógico do atributo IsAuthenticated. Use este método para validar o estado antes de realizar chamadas à API.

3.6 GetLastError()

Retorna a última mensagem de erro registrada em cLastError. Útil para log e diagnóstico de falhas de autenticação.

4. Exemplos de Uso

4.1 Autenticação básica e consumo de outro endpoint

```
#INCLUDE "Protheus.ch"

User Function ExemploAuth()
  Local oAuth := PDAAuthLogin():New()
  Local oRest := FwRest():New(GetMv("BUD_1505UR"))

  If oAuth:IsAuthenticated()
    oRest:SetPath("/api/Pedidos")
    If oRest:Get(oAuth:GetHeader())
      MsgInfo(oRest:GetResult())
    Else
      MsgAlert("Erro na requisição: " + oRest:GetLastError())
    EndIf
  Else
    MsgAlert("Falha na autenticação: " + oAuth:GetLastError())
  EndIf
Return
```

4.2 Renovação de Token

```
User Function ExemploRefresh()
  Local oAuth := PDAAuthLogin():New()

  // ... lógica de negócio ...

  // Quando o token expirar, renovar sem nova autenticação completa:
  If !oAuth:RefreshToken()
    MsgAlert("Falha ao renovar: " + oAuth:GetLastError())
    // Forçar nova autenticação completa se necessário:
    oAuth := PDAAuthLogin():New()
  EndIf
Return
```

4.3 Uso em Rotina com Tratamento de Erros

```
User Function ChamarAPI()
  Local oAuth := PDAAuthLogin():New()
  Local oRest
  Local cResult := ""

  If !oAuth:IsAuthenticated()
    ConOut("Erro de auth: " + oAuth:GetLastError())
    Return ""
  EndIf

  oRest := FwRest():New(GetMv("BUD_1505UR"))
  oRest:SetPath("/api/Estoque")
```

```
If oRest:Get(oAuth:GetHeader())  
    cResult := oRest:GetResult()  
Else  
    ConOut("Erro HTTP: " + oRest:GetLastError())  
EndIf  
  
Return cResult
```

5. Erros e Tratativas

5.1 Tabela de Erros Conhecidos

Mensagem de Erro	Origem	Tratativa Recomendada
Token não retornado pela API	GetToken()	Verificar credenciais (BUD_1505LG / BUD_1505PW). Validar se usuário está ativo na API.
Erro ao autenticar: <detalhe>	GetToken()	Verificar conectividade com a URL BUD_1505UR. Checar se a API está disponível.
Token não renovado pela API	RefreshToken()	O refreshToken expirou. Instanciar novo objeto PDAAuthLogin para autenticação completa.
Erro ao renovar token: <detalhe>	RefreshToken()	Falha de comunicação. Verificar rede e disponibilidade da API, depois tentar novo New().

5.2 Fluxo de Tratamento Recomendado

```

PDAAuthLogin():New()
|
+-- IsAuthenticated() = .F.? --> GetLastError() --> Log + Abortar
|
+-- IsAuthenticated() = .T.? --> Realizar chamadas com GetHeader()
    |
    +-- Token expirado / retorno 401? --> RefreshToken()
        |
        +-- RefreshToken() = .F.? --> Novo New() (reautenticar)
        |
        +-- RefreshToken() = .T.? --> Continuar com GetHeader()
    
```

5.3 Boas Práticas

- Sempre verificar IsAuthenticated() antes de fazer chamadas à API.
- Usar ConOut() ou MsgAlert() para registrar erros via GetLastError() em ambiente de produção.
- Preferir RefreshToken() ao invés de novo New() para evitar tráfego desnecessário de autenticação.
- Não expor credenciais no código-fonte – utilizar sempre os MV_PAR do Configurador (SIGACFG).
- Em jobs/processos automáticos, criar uma nova instância da classe no início de cada execução.

6. Atributos Internos da Classe

Os atributos abaixo são de uso interno, mas podem ser consultados diretamente quando necessário:

Atributo	Tipo	Descrição
cUrlBase	string	URL base da API (lida de BUD_1505UR)
cLogin	string	Login utilizado na autenticação
cPassword	string	Senha utilizada na autenticação
cToken	string	Bearer Token (accessToken) atual – formato JWT
cRefreshToken	string	Refresh Token atual (UUID)
aHeader	array	Array de cabeçalhos HTTP prontos para uso
cEndPoint	string	Último endpoint chamado
cCreated	string	Data/hora de criação do token
cExpiration	string	Data/hora de expiração do token
lAuthenticated	lógico	Indica se o objeto está autenticado (.T./F.)
cLastError	string	Mensagem do último erro ocorrido

7. Histórico de Versões

Versão	Data	Autor	Alterações
1.0	28/01/2026	Caique Silva	Versão inicial – classe PDAAuthLogin com GetToken, GetHeader, RefreshToken, IsAuthenticated e GetLastError.